Orderflow Synthesis Pro [JOAT]

```
// This Pine Script® code is subject to the terms of the Mozilla Public License 2.0 at https://mozilla.org/MPL/2.0/
// © officialjackofalltrades
//@version=6
indicator("Orderflow Synthesis Pro [JOAT]", overlay=true, max_boxes_count=500, max_lines_count=500, max_labels_count=500)

//------------------------------------------------------------------------------
// INPUTS
//------------------------------------------------------------------------------
obLookback = input.int(20, "Order Block Lookback", minval=5, maxval=100, group="Order Block Detection")
obVolMult = input.float(1.3, "Volume Multiplier", minval=1.0, maxval=3.0, group="Order Block Detection")
obAtrMult = input.float(1.5, "ATR Multiplier", minval=1.0, maxval=3.0, group="Order Block Detection")
maxOBs = input.int(8, "Maximum Active Blocks", minval=3, maxval=20, group="Order Block Detection")
obStrengthFilter = input.bool(true, "Strength Classification", group="Order Block Detection")
showOBLabels = input.bool(true, "Display Labels", group="Order Block Detection")

fvgMinSize = input.float(0.3, "Minimum Gap Size (%)", minval=0.1, maxval=2.0, group="Fair Value Gaps")
maxFVGs = input.int(15, "Maximum Active Gaps", minval=5, maxval=30, group="Fair Value Gaps")
fvgAutoFill = input.bool(true, "Auto-Fill Detection", group="Fair Value Gaps")
fvgStrength = input.string("All", "Quality Filter", options=["All", "Strong Only", "Extreme Only"], group="Fair Value Gaps")

liqSwingLen = input.int(5, "Swing Detection Length", minval=3, maxval=15, group="Liquidity Analysis")
liqTolerance = input.float(0.15, "Level Tolerance (%)", minval=0.05, maxval=0.5, group="Liquidity Analysis")
liqShowSweeps = input.bool(true, "Show Sweep Markers", group="Liquidity Analysis")
```

```
liqVolumeConfirm = input.bool(true, "Volume Confirmation", group="Liquidity Analysis")

pdArrayLen = input.int(50, "Array Lookback Period", minval=20, maxval=200, group="Premium
Discount Arrays")
pdShowFib = input.bool(true, "Fibonacci Levels", group="Premium Discount Arrays")
pdDynamicEQ = input.bool(true, "Dynamic Equilibrium", group="Premium Discount Arrays")

// Orderflow Features
showVolumeImbalance = input.bool(true, "Volume Imbalance", group="Advanced Orderflow")
showBreakers = input.bool(true, "Breaker Blocks", group="Advanced Orderflow")
showMitigationBlocks = input.bool(true, "Mitigation Blocks", group="Advanced Orderflow")
showGradientCandles = input×bool(true, "Gradient Candles", group="Visual Settings")

//
─────────────────────────────────────────────────────────────────────────────────
─────────────────────────────────────────────────────────────
// COLOR SCHEME - Iridescent Plasma Gradient
//
─────────────────────────────────────────────────────────────────────────────────
─────────────────────────────────────────────────────────────
// Order Block Colors - Dark with Glowing Borders
COLOR_OB_BULL_FILL = color.new(#0d1117, 70)
COLOR_OB_BULL_BORDER = color.new(#00ffaa, 0)
COLOR_OB_BULL_TEXT = color.new(#00ffaa, 0)
COLOR_OB_BEAR_FILL = color.new(#0d1117, 70)
COLOR_OB_BEAR_BORDER = color.new(#ff0066, 0)
COLOR_OB_BEAR_TEXT = color.new(#ff0066, 0)

// Fair Value Gap Colors - Vibrant Neon
COLOR_FVG_BULL = color.new(#00ff9f, 85)
COLOR_FVG_BULL_BORDER = color.new(#00ff9f, 40)
COLOR_FVG_BEAR = color.new(#ff006e, 85)
COLOR_FVG_BEAR_BORDER = color.new(#ff006e, 40)

// Liquidity Colors - Electric Plasma
COLOR_LIQ_BULL = color.new(#00d4ff, 15)
COLOR_LIQ_BEAR = color.new(#ff0080, 15)
```

```
// Premium/Discount Colors - Multi-color Gradient
COLOR_PREMIUM_1 = color.new(#ff0080, 96)
COLOR_PREMIUM_2 = color.new(#ff4500, 96)
COLOR_DISCOUNT_1 = color.new(#00ced1, 96)
COLOR_DISCOUNT_2 = color.new(#00ff9f, 96)
COLOR_EQUILIBRIUM_FILL = color.new(#1a1a2e, 98)

// Candle Colors - Gradient Bull/Bear
COLOR_CANDLE_BULL_BODY = color.new(#00ffaa, 0)
COLOR_CANDLE_BULL_WICK = color.new(#00d4ff, 0)
COLOR_CANDLE_BEAR_BODY = color.new(#ff0066, 0)
COLOR_CANDLE_BEAR_WICK = color.new(#ff4500, 0)

// Volume Profile Colors
COLOR_HIGH_VOL = color.new(#ffd700, 50)
COLOR_LOW_VOL = color.new(#8b5cf6, 75)

//
————————————————————————————————————————————————————————————————————————————————
// CALCULATIONS
//
————————————————————————————————————————————————————————————————————————————————
atr = ta×atr(14)
avgVol = ta.sma(volume, 20)
volatilityMeasure = ta×atr(200)
atrSma50 = ta.sma(atr, 50)  // Pre-calculate for consistency

// Volume Delta Calculation
buyVolume = close > open ? volume : 0
sellVolume = close < open ? volume : 0
volumeDelta = ta.cum(buyVolume) - ta.cum(sellVolume)
volumeImbalance = volume > avgVol * 1.5 and math.abs(close - open) > atr * 0.5

//
————————————————————————————————————————————————————————————————————————————————
```

```
// GRADIENT CANDLE COLORING
//
-------------------------------------------------------------------------------------------------------------------------

isBullCandle = close > open
candleStrength = math×abs(close - open) / atr

bodyColor = showGradientCandles ?
          (isBullCandle ?
           (candleStrength > 1.5 ? COLOR_CANDLE_BULL_BODY : color.new(#00d4ff, 0)) :
           (candleStrength > 1.5 ? COLOR_CANDLE_BEAR_BODY : color.new(#ff4500, 0))) : na

barcolor(bodyColor)

//
-------------------------------------------------------------------------------------------------------------------------

// VOLUME IMBALANCE DETECTION
//
-------------------------------------------------------------------------------------------------------------------------

if showVolumeImbalance and volumeImbalance
   imbalanceColor = close > open ? color.new(#00ffaa, 90) : color.new(#ff0066, 90)
   label.new(bar_index, high + atr * 0.3, "VOL",
          style=label.style_label_down,
          color=imbalanceColor,
          textcolor=close > open ? #00ffaa : #ff0066,
          size=size×tiny)

//
-------------------------------------------------------------------------------------------------------------------------

// ORDER BLOCKS DETECTION
//
-------------------------------------------------------------------------------------------------------------------------

type OrderBlock
   float top
   float bottom
```

```
    int leftBar
    bool isBullish
    bool mitigated
    box boxObj
    line eqLine
    label textLabel

var array<OrderBlock> orderBlocks = array.new<OrderBlock>()

// Bullish Order Block: Last bearish candle before strong bullish move
bullishOB = close[1] < open[1] and close > open and
            (high - low) > atr * obAtrMult and
            volume > avgVol * obVolMult

if bullishOB
    // Remove oldest if at max
    if array.size(orderBlocks) >= maxOBs
        oldOB = array.shift(orderBlocks)
        box.delete(oldOB.boxObj)
        line.delete(oldOB.eqLine)
        label.delete(oldOB.textLabel)

    // Calculate equilibrium (50% level)
    obTop = high[1]
    obBottom = low[1]
    obEQ = (obTop + obBottom) / 2

    // Create new OB with dark box
    newBox = box.new(bar_index[1], obTop, bar_index + 50, obBottom,
                     border_color=COLOR_OB_BULL_BORDER,
                     bgcolor=COLOR_OB_BULL_FILL,
                     border_width=2,
                     extend=extend×right)

    // Create equilibrium dashed line through middle
    newEQLine = line.new(bar_index[1], obEQ, bar_index + 50, obEQ,
                         color=COLOR_OB_BULL_BORDER,
                         width=1,
```

```
                    style=line.style_dashed,
                    extend=extend×right)

    // Create label inside box at top
    obStrengthText = volume > avgVol * 1.8 ? "STRONG" : "BULL"
    newLabel = label.new(bar_index[1] + 3, obTop - (obTop - obBottom) * 0.15,
                    "OB " + obStrengthText,
                    style=label.style_none,
                    color=color×new(color×black, 100),
                    textcolor=COLOR_OB_BULL_TEXT,
                    size=size×small)

    newOB = OrderBlock.new(obTop, obBottom, bar_index[1], true, false, newBox, newEQLine,
newLabel)
    array.push(orderBlocks, newOB)

// Bearish Order Block: Last bullish candle before strong bearish move
bearishOB = close[1] > open[1] and close < open and
            (high - low) > atr * obAtrMult and
            volume > avgVol * obVolMult

if bearishOB
    if array.size(orderBlocks) >= maxOBs
        oldOB = array.shift(orderBlocks)
        box.delete(oldOB.boxObj)
        line.delete(oldOB.eqLine)
        label.delete(oldOB.textLabel)

    // Calculate equilibrium (50% level)
    obTop = high[1]
    obBottom = low[1]
    obEQ = (obTop + obBottom) / 2

    // Create new OB with dark box
    newBox = box.new(bar_index[1], obTop, bar_index + 50, obBottom,
                    border_color=COLOR_OB_BEAR_BORDER,
                    bgcolor=COLOR_OB_BEAR_FILL,
                    border_width=2,
```

```
                extend=extend×right)

    // Create equilibrium dashed line through middle
    newEQLine = line.new(bar_index[1], obEQ, bar_index + 50, obEQ,
                    color=COLOR_OB_BEAR_BORDER,
                    width=1,
                    style=line.style_dashed,
                    extend=extend.right)

    // Create label inside box at top
    obStrengthText = volume > avgVol * 1.8 ? "STRONG" : "BEAR"
    newLabel = label.new(bar_index[1] + 3, obTop - (obTop - obBottom) * 0.15,
                    "OB " + obStrengthText,
                    style=label.style_none,
                    color=color×new(color×black, 100),
                    textcolor=COLOR_OB_BEAR_TEXT,
                    size=size×small)

    newOB = OrderBlock.new(obTop, obBottom, bar_index[1], false, false, newBox, newEQLine,
newLabel)
    array.push(orderBlocks, newOB)

// Mitigate Order Blocks
if array.size(orderBlocks) > 0
    for i = array×size(orderBlocks) - 1 to 0
        ob = array.get(orderBlocks, i)
        if not ob.mitigated
            if ob.isBullish and close < ob.bottom
                ob.mitigated := true
                box.delete(ob.boxObj)
                line.delete(ob.eqLine)
                label.delete(ob.textLabel)
                array.remove(orderBlocks, i)
            else if not ob.isBullish and close > ob.top
                ob.mitigated := true
                box.delete(ob.boxObj)
                line.delete(ob.eqLine)
                label.delete(ob.textLabel)
```

```
            array.remove(orderBlocks, i)

//
------------------------------------------------------------------------------------------------------------------------
// FAIR VALUE GAPS
//
------------------------------------------------------------------------------------------------------------------------
type FVG
    float top
    float bottom
    int leftBar
    bool isBullish
    bool filled
    box boxObj

var array<FVG> fvgArray = array.new<FVG>()

// Bullish FVG: Gap between candle[2] high and current low
bullishFVG = low > high[2]
fvgBullSize = bullishFVG ? (low - high[2]) / high[2] * 100 : 0

if bullishFVG and fvgBullSize >= fvgMinSize
    if array.size(fvgArray) >= maxFVGs
        oldFVG = array.shift(fvgArray)
        box.delete(oldFVG.boxObj)

    newFVG = FVG.new(
         low,
         high[2],
         bar_index[2],
         true,
         false,
         box.new(bar_index[2], low, bar_index + 30, high[2],
               border_color=color×new(#089981, 60),
               bgcolor=COLOR_FVG_BULL,
               border_width=1)
```

```
    )
    array.push(fvgArray, newFVG)

// Bearish FVG: Gap between candle[2] low and current high
bearishFVG = high < low[2]
fvgBearSize = bearishFVG ? (low[2] - high) / low[2] * 100 : 0

if bearishFVG and fvgBearSize >= fvgMinSize
    if array.size(fvgArray) >= maxFVGs
        oldFVG = array.shift(fvgArray)
        box.delete(oldFVG.boxObj)

    newFVG = FVG.new(
        low[2],
        high,
        bar_index[2],
        false,
        false,
        box.new(bar_index[2], low[2], bar_index + 30, high,
                border_color=color×new(#f23645, 60),
                bgcolor=COLOR_FVG_BEAR,
                border_width=1)
    )
    array.push(fvgArray, newFVG)

// Fill FVGs
if array.size(fvgArray) > 0
    for i = array×size(fvgArray) - 1 to 0
        fvg = array.get(fvgArray, i)
        if not fvg.filled
            if fvg.isBullish and low <= fvg.bottom
                fvg.filled := true
                box.delete(fvg.boxObj)
                array.remove(fvgArray, i)
            else if not fvg.isBullish and high >= fvg.top
                fvg.filled := true
                box.delete(fvg.boxObj)
                array.remove(fvgArray, i)
```

```
//
------------------------------------------------------------------------------------------------------------------------
// LIQUIDITY ZONES (Equal Highs/Lows)
//
------------------------------------------------------------------------------------------------------------------------
pivotHigh = ta×pivothigh(high, liqSwingLen, liqSwingLen)
pivotLow = ta×pivotlow(low, liqSwingLen, liqSwingLen)

var line[] liqLines = array.new_line()
var label[] liqLabels = array.new_label()

// Equal Highs (Sell-side liquidity)
if not na(pivotHigh)
    isEqual = false
    for i = 1 to 3
        if not na(pivotHigh[i * liqSwingLen])
            diff = math.abs(pivotHigh - pivotHigh[i * liqSwingLen]) / pivotHigh * 100
            if diff <= liqTolerance
                isEqual := true
                break

    if isEqual
        if array.size(liqLines) > 20
            line.delete(array.shift(liqLines))
            label.delete(array.shift(liqLabels))

        liqLine = line.new(bar_index[liqSwingLen], pivotHigh, bar_index + 30, pivotHigh,
                           color=COLOR_LIQ_BEAR, width=3, style=line.style_solid)
        array.push(liqLines, liqLine)

        liqLabel = label.new(bar_index + 15, pivotHigh, "SELL LIQ",
                             style=label.style_label_down,
                             color=COLOR_LIQ_BEAR,
                             textcolor=color×white,
                             size=size×small)
```

```
        array.push(liqLabels, liqLabel)

        if liqShowSweeps
            // Draw liquidity sweep marker
            box.new(bar_index[liqSwingLen] - 5, pivotHigh + atr * 0.2,
                    bar_index[liqSwingLen] + 5, pivotHigh - atr * 0.2,
                    border_color=COLOR_LIQ_BEAR,
                    bgcolor=color.new(COLOR_LIQ_BEAR, 85),
                    border_width=1)

// Equal Lows (Buy-side liquidity)
if not na(pivotLow)
    isEqual = false
    for i = 1 to 3
        if not na(pivotLow[i * liqSwingLen])
            diff = math.abs(pivotLow - pivotLow[i * liqSwingLen]) / pivotLow * 100
            if diff <= liqTolerance
                isEqual := true
                break

    if isEqual
        if array.size(liqLines) > 20
            line.delete(array.shift(liqLines))
            label.delete(array.shift(liqLabels))

        liqLine = line.new(bar_index[liqSwingLen], pivotLow, bar_index + 30, pivotLow,
                           color=COLOR_LIQ_BULL, width=3, style=line.style_solid)
        array.push(liqLines, liqLine)

        liqLabel = label.new(bar_index + 15, pivotLow, "BUY LIQ",
                             style=label.style_label_up,
                             color=COLOR_LIQ_BULL,
                             textcolor=color×white,
                             size=size×small)
        array.push(liqLabels, liqLabel)

        if liqShowSweeps
            // Draw liquidity sweep marker
```

```
            box.new(bar_index[liqSwingLen] - 5, pivotLow + atr * 0.2,
                bar_index[liqSwingLen] + 5, pivotLow - atr * 0.2,
                border_color=COLOR_LIQ_BULL,
                bgcolor=color.new(COLOR_LIQ_BULL, 85),
                border_width=1)

//
------------------------------------------------------------------------------------------------------------------------
// PREMIUM / DISCOUNT ARRAYS
//
------------------------------------------------------------------------------------------------------------------------
rangeHigh = ta×highest(high, pdArrayLen)
rangeLow = ta×lowest(low, pdArrayLen)
rangeEQ = (rangeHigh + rangeLow) / 2

// Premium zone (above equilibrium)
inPremium = close > rangeEQ and close > (rangeEQ + (rangeHigh - rangeEQ) * 0.5)
// Discount zone (below equilibrium)
inDiscount = close < rangeEQ and close < (rangeEQ - (rangeEQ - rangeLow) * 0.5)

// Multi-gradient background based on position
bgGradient = if inPremium
    distanceFromEQ = (close - rangeEQ) / (rangeHigh - rangeEQ)
    color.from_gradient(distanceFromEQ, 0.5, 1.0, COLOR_PREMIUM_2, COLOR_PREMIUM_1)
else if inDiscount
    distanceFromEQ = (rangeEQ - close) / (rangeEQ - rangeLow)
    color.from_gradient(distanceFromEQ, 0.5, 1.0, COLOR_DISCOUNT_2, COLOR_DISCOUNT_1)
else
    COLOR_EQUILIBRIUM_FILL

bgcolor(bgGradient, title="PD Array")

// Iridescent Equilibrium Line - Multi-color blend
eqColor1 = color.new(#9370db, 0)  // Purple
eqColor2 = color.new(#00d4ff, 0)  // Cyan
eqColor3 = color.new(#ff0080, 0)  // Magenta
```

```
eqColor4 = color.new(#00ffaa, 0)  // Mint

// Create glowing effect with multiple plots
plot(rangeEQ, "EQ Glow 1", color.new(eqColor1, 70), 6, plot.style_line)
plot(rangeEQ, "EQ Glow 2", color.new(eqColor2, 60), 4, plot.style_line)
plot(rangeEQ, "EQ Core", color.from_gradient(bar_index % 100, 0, 100, eqColor3, eqColor4), 2,
plot.style_line)

// Add Fibonacci-style levels for value array
premium75 = rangeEQ + (rangeHigh - rangeEQ) * 0.75
premium50 = rangeEQ + (rangeHigh - rangeEQ) * 0.5
discount50 = rangeEQ - (rangeEQ - rangeLow) * 0.5
discount75 = rangeEQ - (rangeEQ - rangeLow) * 0.75

plot(pdShowFib ? rangeHigh : na, "Range High", color.new(#ff0080, 50), 2, plot.style_stepline)
plot(pdShowFib ? premium75 : na, "Premium 75%", color.new(#ff4500, 60), 1, plot.style_stepline)
plot(pdShowFib ? premium50 : na, "Premium 50%", color.new(#ff7f50, 70), 1, plot.style_stepline)
plot(pdShowFib ? discount50 : na, "Discount 50%", color.new(#00ced1, 70), 1, plot.style_stepline)
plot(pdShowFib ? discount75 : na, "Discount 75%", color.new(#00d4ff, 60), 1, plot.style_stepline)
plot(pdShowFib ? rangeLow : na, "Range Low", color.new(#00ffaa, 50), 2, plot.style_stepline)

//
------------------------------------------------------------------------------------------
------------------------------------------------------------
// CONFLUENCE SCORING
//
------------------------------------------------------------------------------------------
------------------------------------------------------------
confluenceScore = 0
confluenceScore += array.size(orderBlocks) > 3 ? 25 : array.size(orderBlocks) > 1 ? 15 : 5
confluenceScore += array.size(fvgArray) > 5 ? 25 : array.size(fvgArray) > 2 ? 15 : 5
confluenceScore += array.size(liqLines) > 3 ? 20 : array.size(liqLines) > 1 ? 10 : 0
confluenceScore += inPremium or inDiscount ? 15 : 5
volRatio = atr / ta.sma(atr, 50)
confluenceScore += volRatio > 1.3 ? 15 : volRatio < 0.8 ? 10 : 5

confluenceLevel = confluenceScore >= 80 ? "EXTREME" : confluenceScore >= 60 ? "STRONG" :
confluenceScore >= 40 ? "MODERATE" : "WEAK"
```

```
confluenceColor = confluenceScore >= 80 ? color.new(#00ff00, 0) : confluenceScore >= 60 ?
color.new(#00d4ff, 0) : confluenceScore >= 40 ? color.new(#ffd700, 0) : color.gray

//
------------------------------------------------------------------------------
// BREAKER BLOCKS (Failed Order Blocks that reverse)
//
------------------------------------------------------------------------------
var box[] breakerBlocks = array.new_box()
var label[] breakerLabels = array.new_label()

// Detect breaker: OB gets broken and price reverses
if showBreakers and array.size(orderBlocks) > 0
    for i = 0 to array.size(orderBlocks) - 1
        ob = array.get(orderBlocks, i)

        // Bullish OB broken to downside = Bearish Breaker
        if ob.isBullish and close < ob.bottom and close[1] >= ob.bottom
            if array.size(breakerBlocks) > 10
                box.delete(array.shift(breakerBlocks))
                label.delete(array.shift(breakerLabels))

            breakerBox = box.new(bar_index - 3, ob.top, bar_index + 20, ob.bottom,
                         border_color=color×new(#ff0066, 30),
                         bgcolor=color.new(#ff0066, 92),
                         border_width=1,
                         border_style=line.style_dotted)
            array.push(breakerBlocks, breakerBox)

            breakerLabel = label.new(bar_index, ob.top, "BREAKER",
                           style=label.style_label_down,
                           color=color×new(#ff0066, 70),
                           textcolor=#ff0066,
                           size=size×tiny)
            array.push(breakerLabels, breakerLabel)
```

```
        // Bearish OB broken to upside = Bullish Breaker
        if not ob.isBullish and close > ob.top and close[1] <= ob.top
            if array.size(breakerBlocks) > 10
                box.delete(array.shift(breakerBlocks))
                label.delete(array.shift(breakerLabels))

            breakerBox = box.new(bar_index - 3, ob.top, bar_index + 20, ob.bottom,
                                 border_color=color.new(#00ffaa, 30),
                                 bgcolor=color.new(#00ffaa, 92),
                                 border_width=1,
                                 border_style=line.style_dotted)
            array.push(breakerBlocks, breakerBox)

            breakerLabel = label.new(bar_index, ob.bottom, "BREAKER",
                                     style=label.style_label_up,
                                     color=color.new(#00ffaa, 70),
                                     textcolor=#00ffaa,
                                     size=size.tiny)
            array.push(breakerLabels, breakerLabel)

//
------------------------------------------------------------------------------------------------------------------------
// MITIGATION BLOCKS (Price returns to OB for retest)
//
------------------------------------------------------------------------------------------------------------------------
if showMitigationBlocks and array.size(orderBlocks) > 0
    for i = 0 to array.size(orderBlocks) - 1
        ob = array.get(orderBlocks, i)

        // Price returns to bullish OB
        if ob.isBullish and low <= ob.top and low >= ob.bottom and close > open
            label.new(bar_index, low, "MIT",
                      style=label.style_label_up,
                      color=color.new(#00ffaa, 80),
                      textcolor=#00ffaa,
                      size=size.tiny)
```

```
        // Price returns to bearish OB
        if not ob.isBullish and high >= ob.bottom and high <= ob.top and close < open
            label.new(bar_index, high, "MIT",
                      style=label.style_label_down,
                      color=color×new(#ff0066, 80),
                      textcolor=#ff0066,
                      size=size×tiny)

//
——————————————————————————————————————————————————————————————————————————————
// DASHBOARD
//
——————————————————————————————————————————————————————————————————————————————
if barstate.islast
    var table dashboard = table.new(position.top_right, 2, 8,
                                    bgcolor=color.new(#0a0a0a, 5),
                                    border_width=3,
                                    border_color=color×new(#00d4ff, 40))

    table.cell(dashboard, 0, 0, "LIQUIDITY FLOW [JOAT]", text_color=color×new(#00d4ff, 0),
text_size=size×normal, bgcolor=color.new(#1a1a2e, 20))
    table.merge_cells(dashboard, 0, 0, 1, 0)

    table.cell(dashboard, 0, 1, "Confluence", text_color=color.new(#a0a0a0, 0), text_size=size×small)
    table.cell(dashboard, 1, 1, confluenceLevel, text_color=confluenceColor, text_size=size×small)

    table.cell(dashboard, 0, 2, "Order Blocks", text_color=color.new(#a0a0a0, 0),
text_size=size×small)
    obCount = str×tostring(array×size(orderBlocks))
    obColor = array×size(orderBlocks) > 5 ? color.new(#00ff88, 0) : array.size(orderBlocks) > 2 ?
color.new(#ffd700, 0) : color.gray
    table.cell(dashboard, 1, 2, obCount + " Active", text_color=obColor, text_size=size×small)

    table.cell(dashboard, 0, 3, "Imbalance Zones", text_color=color.new(#a0a0a0, 0),
text_size=size×small)
```

```
    fvgCount = str.tostring(array.size(fvgArray))
    fvgColor = array.size(fvgArray) > 8 ? color.new(#00ffaa, 0) : array.size(fvgArray) > 4 ?
color.new(#ffd700, 0) : color.gray
    table.cell(dashboard, 1, 3, fvgCount + " Open", text_color=fvgColor, text_size=size.small)

    table.cell(dashboard, 0, 4, "Liquidity Pools", text_color=color.new(#a0a0a0, 0),
text_size=size.small)
    liqCount = str.tostring(array.size(liqLines))
    liqColor = array.size(liqLines) > 5 ? color.new(#ff0066, 0) : array.size(liqLines) > 2 ?
color.new(#ffd700, 0) : color.gray
    table.cell(dashboard, 1, 4, liqCount + " Zones", text_color=liqColor, text_size=size.small)

    table.cell(dashboard, 0, 5, "Value Array", text_color=color.new(#a0a0a0, 0),
text_size=size.small)
    arrayPos = inPremium ? "PREMIUM" : inDiscount ? "DISCOUNT" : "EQUILIBRIUM"
    arrayColor = inPremium ? color.new(#ff4500, 0) : inDiscount ? color.new(#00ced1, 0) :
color.new(#9370db, 0)
    table.cell(dashboard, 1, 5, arrayPos, text_color=arrayColor, text_size=size.small)

    table.cell(dashboard, 0, 6, "Flow State", text_color=color.new(#a0a0a0, 0), text_size=size.small)
    volState = atr > atrSma50 * 1.5 ? "EXPLOSIVE" : atr < atrSma50 * 0.7 ? "DORMANT" : "ACTIVE"
    volColor = atr > atrSma50 * 1.5 ? color.new(#ff0066, 0) : atr < atrSma50 * 0.7 ? color.gray :
color.new(#00d4ff, 0)
    table.cell(dashboard, 1, 6, volState, text_color=volColor, text_size=size.small)
    table.cell(dashboard, 1, 6, volState, text_color=volColor, text_size=size.small)

    table.cell(dashboard, 0, 7, "Signal Quality", text_color=color.new(#a0a0a0, 0),
text_size=size.small)
    signalQuality = str.tostring(confluenceScore) + "/100"
    table.cell(dashboard, 1, 7, signalQuality, text_color=confluenceColor, text_size=size.small)

//
------------------------------------------------------------------------------------------
-------------------------------------------------------------
// ALERTS
//
------------------------------------------------------------------------------------------
-------------------------------------------------------------
```

```
alertcondition(bullishOB, "Bullish Order Block", "New Bullish Order Block Formed")
alertcondition(bearishOB, "Bearish Order Block", "New Bearish Order Block Formed")
alertcondition(bullishFVG, "Bullish FVG", "New Bullish Fair Value Gap")
alertcondition(bearishFVG, "Bearish FVG", "New Bearish Fair Value Gap")
```